

บทที่ 2

เทคโนโลยี .net

2.1 ความเป็นมาของ .net

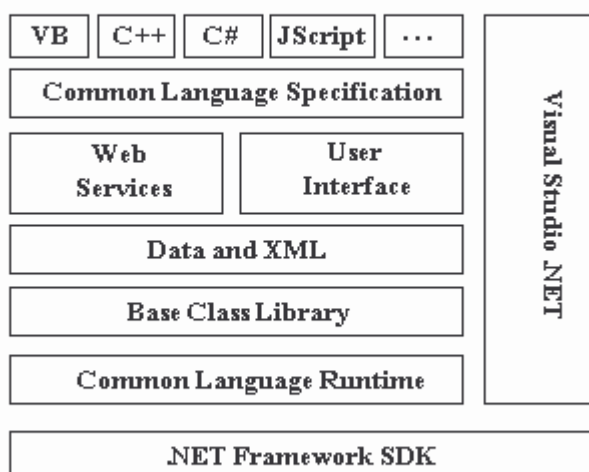
เนื่องจากปัญหาอย่างหนึ่งที่เกิดขึ้นกับอินเทอร์เน็ตก็คือ เครื่องพีซี และ อุปกรณ์สื่อสารต่างๆ รวมทั้งเซิร์ฟเวอร์จำนวนมากที่ประกอบกันเป็นเครือข่ายขนาดใหญ่ ยังขาดวิธีแลกเปลี่ยนข้อมูลกันได้อย่างสมบูรณ์ ทำให้ผู้ใช้ไม่สามารถควบคุมข้อมูลและเทคโนโลยีที่มีอยู่รอบตัวได้ โดยข้อมูลที่สำคัญมักถูกเก็บไว้อย่างกระจัดกระจายในแอปพลิเคชัน และเว็บไซต์ต่างๆ ซึ่งยังขาดวิธีที่มีประสิทธิภาพในการนำข้อมูลเหล่านั้นมาใช้งานร่วมกันได้

หลักการของ .net คือ แนวคิดในการทำให้เว็บไซต์และโปรแกรมต่างๆ ที่ทำงานบนเครื่องพีซี และอุปกรณ์ต่างๆ ที่อาจทำงานแยกจากกันในอดีต ให้สามารถสื่อสารและแลกเปลี่ยนข้อมูลระหว่างกันได้ โดยจะสร้างซอฟต์แวร์ที่เป็นบริการ หรือเว็บเซอร์วิส ไว้ในผลิตภัณฑ์ต่างๆ ของไมโครซอฟท์ ตั้งแต่ windows ที่ใช้กับเครื่องพีซีไปจนถึง windows ที่ใช้กับเครื่องเซิร์ฟเวอร์ และนำไปผนวกกับเว็บไซต์ต่างๆ ด้วย ซึ่งไมโครซอฟท์เล็งเห็นว่าในที่สุดผู้คนจะหันมาใช้ เว็บเซอร์วิส แทนแอปพลิเคชันต่างๆ แบบเดิมบนเครื่องพีซี แนวคิดนี้จะนำไปไม่ได้ถ้าขาด XML ซึ่งเป็นภาษามาตรฐานในการแลกเปลี่ยนข้อมูล โดยเว็บเซอร์วิสต่างๆ จะใช้สื่อสารระหว่างกัน โดยทาง ไมโครซอฟท์เชื่อว่า XML จะเชื่อมโลกของข้อมูลเข้าด้วยกันได้ เราสามารถสรุปได้ว่า .net ก็คือแพลตฟอร์ม (Platform) ของไมโครซอฟท์ สำหรับการบริการเว็บแบบ XML เว็บเซอร์วิส (XML web service) เป็นซอฟต์แวร์รุ่นต่อไปที่ใช้สำหรับการติดต่อในโลกของข้อมูล และอุปกรณ์สื่อสารรูปแบบต่างๆ เข้าด้วยกัน ถือเป็นการเปลี่ยนแปลงครั้งสำคัญในการใช้งานแอปพลิเคชันผ่านเว็บและอุปกรณ์ของผู้ใช้ แพลตฟอร์ม .net ช่วยให้การสร้างแอปพลิเคชันทั่วไป และเว็บแอปพลิเคชันที่มีอยู่มากมายเป็นเหมือนกับบริการที่สามารถมีการใช้งานและรวมข้อมูลต่างๆ ไว้ด้วยกัน เพื่อให้บริการผ่านทุกอุปกรณ์ที่ทำงานกับเว็บได้ โดยการใช้ภาษา XML เป็นมาตรฐาน .net เป็นนิยามของการให้บริการซอฟต์แวร์ในรูปแบบของ service ซึ่งจะรันโดยไม่ขึ้นอยู่กับอุปกรณ์เชื่อมต่อ หรือระบบปฏิบัติการใดๆ เลย ตามที่ไมโครซอฟท์กล่าวว่า สามารถใช้บริการได้ทุกเวลา ทุกแห่ง และทุกอุปกรณ์ คือเราสามารถใช้อุปกรณ์ไม่ว่าจะเป็น โทรศัพท์มือถือ, PDA หรือเครื่องคอมพิวเตอร์พีซีทั่วไป ในการติดต่อสื่อสาร ทำงานกับเซอร์วิส ผ่านอินเทอร์เน็ตได้เหมือนกันหมด ซึ่งการจะทำให้สิ่งนี้สำเร็จได้นั้น ก็มาจากมาตรฐานใหม่อย่าง XML ภาษามาตรฐานที่สามารถทำงานร่วมกันผ่านแพลตฟอร์มที่แตกต่างกันได้ และ SOAP มาตรฐานการเชื่อมต่อให้สามารถแลกเปลี่ยนข้อมูลกันระหว่างแพลตฟอร์มได้

2.2 สถาปัตยกรรมของ .net

เราทราบแล้วว่า .net เป็นนิยามการให้บริการของซอฟต์แวร์ในรูปแบบของเซอร์วิส ซึ่งจะรันได้โดยไม่ขึ้นอยู่กับอุปกรณ์ที่ใช้แสดงผล หรือระบบปฏิบัติการใดๆ ในส่วนนี้จะกล่าวถึงโครงสร้าง

โดยรวมทั้งหมดของการสร้างแอปพลิเคชัน .net (.net Application Architecture) ซึ่งแสดงดังรูปที่ 2-1

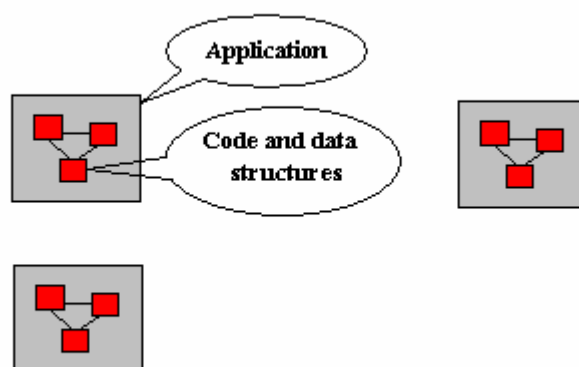


รูปที่ 2-1 โครงสร้างการพัฒนาแอปพลิเคชัน .net

รูปที่ 2-1 แสดงถึงสถาปัตยกรรมของแอปพลิเคชัน .net ที่พัฒนาด้วย Visual Studio.net ถือว่าเป็นการเปลี่ยนแปลงครั้งสำคัญ ทั้งแพลตฟอร์มที่ใช้ในการพัฒนาแอปพลิเคชัน และสถาปัตยกรรมที่ใช้ โดยมีเลเยอร์ล่างสุดคือ .net Framework SDK เปรียบเสมือน Runtime Library ที่จะรันอยู่ คอยสนับสนุนการทำงานของแอปพลิเคชัน จากนั้นจะเป็นเลเยอร์ของ Common Language Runtime เป็นผลลัพธ์ของการคอมไพล์ (compile) แอปพลิเคชัน .net เลเยอร์ถัดขึ้นมาเป็นเครื่องมือ (Tools) และเทคนิคต่างๆ ที่เราสามารถใช้ในการพัฒนาแอปพลิเคชัน ได้ทั้งในเรื่องของ web service, ADO.net, ASP.net จนกระทั่งถึงเลเยอร์บนสุดคือภาษาที่ใช้ในการพัฒนาแอปพลิเคชันด้วย Visual Studio.net

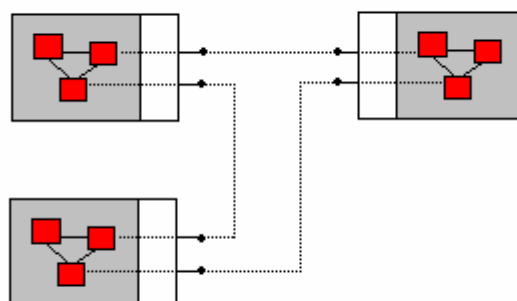
2.3 เลเยอร์ Common Language Runtime

ก่อนที่จะมีการพัฒนาโปรแกรมเป็น Object Oriented นั้น แอปพลิเคชันแต่ละตัวเปรียบเสมือนกล่อง ภายในแอปพลิเคชันก็จะมีโค้ด (code) มีโครงสร้างข้อมูล (data structure) ต่างๆ ของตัวเอง มีฟังก์ชันต่างๆ ของตัวแอปพลิเคชันนั้นๆ ดังแสดงในรูปที่ 2-2



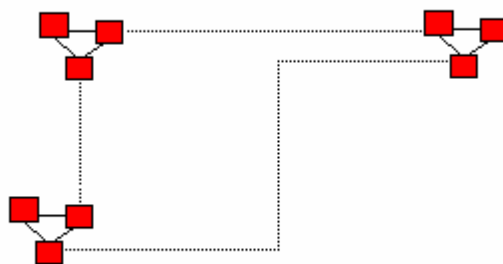
รูปที่ 2-2 โครงสร้างแอปพลิเคชันแรกๆ การติดต่อกันระหว่างแอปพลิเคชันเป็นเรื่องยาก

การที่แอปพลิเคชันต่างๆ จะมีการเรียกใช้การทำงานของกันและกัน หรือมีการส่งผ่านข้อมูลถึงกันและกัน เป็นสิ่งที่ทำได้ยาก ซึ่งอาจต้องมีการกำหนดอะไรขึ้นมาเองระหว่าง 2 แอปพลิเคชันนั้นๆ จนกระทั่ง ในยุคถัดมา ไมโครซอฟท์ได้คิดค้นเทคโนโลยี COM (Component Object Model) เป็นวิธีที่ทำให้เราเขียนโปรแกรมเป็นแบบ Object Oriented และเรียกใช้การทำงานที่มาจากต่างแอปพลิเคชันได้ ดังแสดงในรูปที่ 2-3



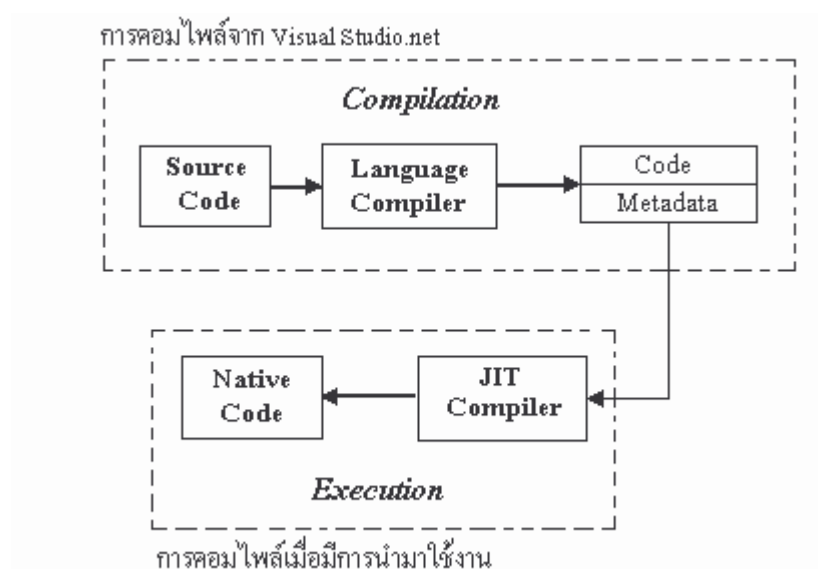
รูปที่ 2-3 แสดงการพัฒนาแอปพลิเคชันแบบ COM ที่สามารถเรียกใช้ฟังก์ชันของแอปพลิเคชันอื่นได้

หากจะอธิบายให้ง่ายขึ้น ก็คือเปรียบเทียบเราเอาแพ็คเกจอันหนึ่งห่อแอปพลิเคชันของเราไว้ และการพูดคุยกันของแอปพลิเคชันก็พูดคุยผ่านแพ็คเกจที่เราห่อเอาไว้ จนกระทั่งมาถึงตัว Visual Studio.net ที่ได้รับการออกแบบใหม่ จะเห็นว่าจากรูปเดิม กล่องแพ็คเกจหายไป ถ้าเราพัฒนาด้วยรูปแบบเทคโนโลยี .net นั้น คลาสต่างๆ สามารถติดต่อกันได้โดยตรงดังรูปที่ 2-4



รูปที่ 2-4 การพัฒนาแอปพลิเคชันบนเทคโนโลยี *.net* คลาสภายในแต่ละแอปพลิเคชันสามารถติดต่อกันได้โดยตรง

การพัฒนาแอปพลิเคชันด้วย Visual Studio.net นั้น เมื่อเราคอมไพล์สิ่งที่เราได้ จะไม่ใช่โค้ดไบนารี (Binary code) เลยทีเดียว แต่จะได้เป็นภาษากลางอันหนึ่งเรียกว่า Microsoft Intermediate Language (MSIL) ซึ่งเป็นภาษาในระดับเลเยอร์ต่างๆ โครงสร้างของภาษา (Syntax) จะเหมือนภาษา Assembly ภายในสิ่งที่เกิดจากการคอมไพล์ก็จะเป็น MSIL ตัวนี้ ภายในตัวมันจะประกอบด้วย 2 ส่วน คือ โค้ด กับตัวแอตทริบิวต์หรือพรีอเพอเรเตอร์ต่างๆ ที่ใช้อธิบายตัวโค้ดนั้น ซึ่งเรียกว่า Metadata ดังรูปที่ 2-5

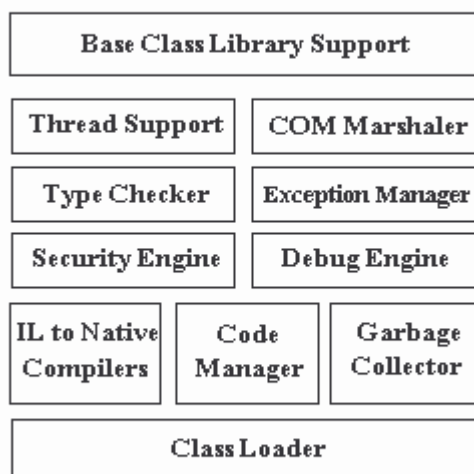


รูปที่ 2-5 โครงสร้างการคอมไพล์จาก Visual Studio .net และเมื่อมีการนำไปใช้งานจริง

จากนั้น เมื่อโค้ดซึ่งเป็น Intermediate Language ถูกเรียกใช้งานจริงๆ จะมีตัว Compiler (ตัวแปลภาษา) ตัวหนึ่งมาทำการคอมไพล์โค้ดตัวนั้นให้เป็นโค้ดไบนารี ซึ่ง Compiler ตัวนั้นจะเรียกว่า Just In Time Compiler (JIT Compiler) ที่เรียกว่า Just In Time เพราะว่าจะมีการคอมไพล์เมื่อมี

การใช้งาน ฉะนั้นคลาสหรือโค้ดต่างๆ ที่เราพัฒนาขึ้นแล้วจะถูกคอมไพล์มาเป็น Intermediate Language ที่มีโครงสร้างของภาษาแบบเดียวกัน เพราะฉะนั้นคลาสต่างๆ ในแอปพลิเคชันจึงสามารถทำงานได้อย่างกลมกลืนกันและไม่มีข้อขัดข้องอะไร

การทำงานของ Common Language Runtime นั้นภายในตัว Common Language Runtime จะมีโมดูล (Module) ย่อยๆ ซึ่งเป็นสถาปัตยกรรมภายในดังรูปที่ 2-6

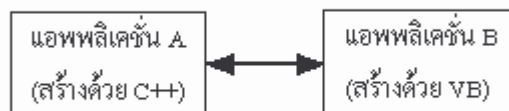


รูปที่ 2-6 แสดงสถาปัตยกรรมของ *Common Language Runtime*

ด้านล่างสุดในรูปที่ 2-6 จะมี Class Loader ซึ่งเอาไว้อโหลดโปรแกรมของเราขึ้นมาทำงาน นอกจากนี้ก็จะมี compiler ซึ่งจะทำการคอมไพล์ภาษา Intermediate Language ให้เป็นภาษาไบนารีโดยจะมีตัว Code Manager และ Garbage Collector คอยจัดการกับหน่วยความจำที่เราจองเวลาเรียกใช้งาน นอกจากนี้ก็จะมีเรื่องของระบบความปลอดภัย (Security) ในการทำงาน รวมทั้ง Debug Engine ในการดัก Runtime Error และตัว Exception Manager การตรวจเช็คชนิดของตัวแปรต่างๆ และด้านบนสุดจะเป็นการใช้งานร่วมกับ Library Class ต่างๆ ซึ่งจัดเตรียมมาให้ เพราะฉะนั้นเลขอร์ของ Common Language Runtime การคอมไพล์แอปพลิเคชันใดก็ตาม ไม่ว่าจะเป็น ASP.net หรือเขียนแอปพลิเคชันบน Window ธรรมดาหรือจะเป็นการเขียน Web service ก็ตาม สิ่งที่ได้จากการคอมไพล์จะเป็น Common Language Runtime

นั่นคือการออกแบบเพื่อสนับสนุนการทำงานร่วมกันใน Common Language Runtime จึงจัดข้อเสียของ COM ไปได้ เนื่องจากข้อจำกัดของ COM คือเป็นเพียงการเอาอะไรบางอย่างมาห่อคลาสเอาไว้เท่านั้น ดังนั้นเวลาที่เอา COM ไปใช้งานจึงค่อนข้างยุ่งยาก รวมทั้งถ้ามีการเปลี่ยนแปลง COM นั้นๆ ก็จะทำให้เกิดปัญหาเรื่องของการเข้ากันได้ (Compatibility) ระหว่างเวอร์ชันเดิมกับเวอร์ชันปัจจุบัน แต่ถ้าเราพัฒนาด้วย Visual Studio.net ข้อเสียของ COM ทั้งหมดก็จะถูกขจัดเสีย

ความจริง Common Language Runtime เป็นหลักการทำงานที่มีวิวัฒนาการมาจาก COM อีกทีหนึ่ง เป็นการทำให้ Object Oriented ที่แกนของภาษาเลย โดย Visual Studio.net นั้นถูกออกแบบเพื่อสนับสนุน Object Oriented โดยเฉพาะ คลาสต่างๆที่อยู่ในแต่ละแอปพลิเคชันสามารถติดต่อถึงกันได้โดยตรง (Inherit) ในแอปพลิเคชัน A เราอาจจะเขียนคลาสด้วยภาษา C++ และแอปพลิเคชัน A อาจจะติดต่อกับแอปพลิเคชัน B ซึ่งเขียนด้วยภาษา Visual Basic (VB) ได้ คือการติดต่อกัน (Inherit) ข้ามภาษานั้นสามารถทำได้ดังแสดงในรูปที่ 2-7



รูปที่ 2-7 ความสามารถในการติดต่อข้ามแอปพลิเคชันเมื่อเขียนด้วยภาษาที่ต่างกันใน Visual Studio .net

ใน Visual Studio.net จะคอมไพล์เป็นภาษาเดียวกันคือ Intermediate Language ตามรูปก่อนหน้านี้ คือคอมไพล์เป็นภาษาอันหนึ่ง เพราะฉะนั้นจึงสามารถ Inherit กันข้ามภาษาได้ นอกจากนี้ยังสามารถทำงานด้วนกันกับ COM แบบเดิมที่เราเคยเขียนมาแล้วได้ด้วย ซึ่งใช้ Visual Studio เดิมโค้ดเหล่านี้ก็ไม่จำเป็นต้องโยนทิ้ง ใน Visual Studio.net เราสามารถเรียกใช้งาน COM ได้ตามปกติและในทางกลับกัน ใน Visual Studio 6.0 ที่มีอยู่ก็สามารถเรียกใช้งานคอมโพเนนต์ที่เขียนด้วย Visual Studio .net ได้เช่นกัน คือเป็นการเข้ากันได้ทั้งสองทาง (Backward-Forward Compatibility) นี่คือข้อดีมากๆของ .net ทำให้เราไม่ต้องมาพัฒนาโค้ดใหม่

2.3.1 การจัดการกับหน่วยความจำเมื่อทำการประมวลผล

เนื่องจากการทำงานทั้งหมดของตัว .net จะมีการดูแลในเรื่องของ Common Language Runtime ปัญหาหนึ่งที่มักจะพบ เมื่อเราพัฒนาแอปพลิเคชัน คือเรื่องของ Garbage Collection ในการจัดการกับหน่วยความจำ โดยเฉพาะบางภาษาอย่างเช่น ภาษา C++ ที่ต้องมีการเรียกใช้งาน Pointer ก่อนข้างมาก ในการใช้งาน Pointer นั้นถ้าเราเขียนผิดไปนิดเดียวก็อาจจะทำให้แอปพลิเคชันของเราแฉงค์ในระหว่างรันหรือประมวลผลก็ได้ ทำให้เมื่อแอปพลิเคชันของเรารันไปเรื่อยๆหน่วยความจำส่วนนี้ก็จะขยายไปเรื่อยๆและสักพักหนึ่งก็จะเกิดการหยุดไป

การแก้ไขข้อผิดพลาด (Debug) ตรงนี้ทำได้ยากมาก เพราะว่ากว่าเราจะรู้ว่าจะเกิดความผิดพลาดขึ้นเราก็เสียเวลาในการรันไปแล้ว ซึ่งการจัดการกับหน่วยความจำตรงนี้ ถ้าเราพัฒนาด้วยตัวแพลตฟอร์มของ .net จะมีเครื่องมือตัวหนึ่งเรียกว่า Garbage Collector เป็นตัวที่คอยจัดการกับหน่วยความจำให้เรา ความจริงตัว Garbage Collector จะเป็นตัวที่ทำการกำหนดค่าหน่วยความจำให้ว่างให้เราเองในส่วนที่เราไม่ใช้งานโดยอัตโนมัติ นี่คือข้อดีอันหนึ่งในการพัฒนาด้วยแพลตฟอร์มของ .net

2.3.2 ระบบการตรวจจับความผิดพลาด

นอกจากนี้ในเรื่องของ Exception Manager หรือการดัก Runtime error ในการสร้างแอปพลิเคชันที่มีการรันได้อย่างดีนั้น การดัก Runtime error ที่จะเกิดขึ้นในระหว่างทำงาน ก็เป็นปัจจัยหนึ่งที่เราจะต้องคำนึงถึง ซึ่งในตัว .net จะมี Syntax ในการดัก Runtime error ที่มีลักษณะเป็นโครงสร้างทำให้หาได้ง่ายขึ้น เราเรียกว่า Structure Exception Handling

2.3.3 รูปแบบการคอมไพล์แอปพลิเคชัน

นอกจากนี้การคอมไพล์แอปพลิเคชันด้วย .net มีให้เลือกหลายแบบ บางคนอาจจะคอมไพล์ให้เป็นไค้ดไบนารี (Binary code) เลยโดยไม่ต้องคอมไพล์ให้เป็น Intermediate Language ก็สามารถพัฒนาได้ โดยใช้ Visual C++ .net เรียกว่า Managed C++ หรือถ้าเราเลือกใช้เครื่องมือภาษาอื่นๆที่มีความง่ายในการพัฒนายิ่งกว่าคือ VB หรือ C# ก็สามารถคอมไพล์เป็น Intermediate Language ได้

ในการคอมไพล์จะไม่มีตัว Interpreter Runtime Library ต่างๆเช่น VB ที่ต้องมี Runtime Library ของตัวเองเวลาพัฒนาด้วย Visual Studio 6.0 อันนี้ก็จะไม่จำเป็นต้องมีอีกต่อไป เราใช้ตัว Runtime อันเดียวกัน ก็คือตัว .net Framework เป็นตัว Runtime Library ที่กล่าวมาข้างต้น

2.3.4 เรื่องของชนิดตัวแปร

ภาษาทุกตัวที่เขียนในตัว .net จะเป็นภาษาแบบ Type-Safety คือการเปลี่ยนแปลงระหว่างค่าชนิดต่างๆ เป็น String type ตัว Compiler จะตรวจสอบได้ ถ้าเรามีการติดต่อกันระหว่างตัวแปร 2 ตัว ที่มีชนิดต่างกันเช่น เราอาจต้องการเปลี่ยนชนิดของค่าในตัวแปรจาก string เป็น integer เป็นต้น ตัวภาษาใน visual Studio 6.0 จะให้เราทำได้ แต่ใน .net จะมีการตรวจสอบว่าถ้ามีการเปลี่ยนค่าจาก string เป็น integer หรือการเปลี่ยนชนิดของตัวแปรอื่นๆที่จะทำให้เกิด Runtime error นั้น ตัว compiler จะตรวจสอบได้และก็จะมีการแจ้งข้อผิดพลาดให้ทราบ

แม้แต่เรื่องของตัวแปรที่เอาไปใช้งาน ถ้าเราประกาศตัวแปรขึ้นมา แล้วเราเอาไปใช้งานโดยไม่ได้มีการกำหนดค่าให้มันก่อน Compiler ก็จะคอยดัก นั่นคือ compiler มีความฉลาดมากขึ้น เพื่อดักข้อผิดพลาด ณ จุดที่คอมไพล์เลย ซึ่งดีกว่าให้เกิด Runtime error ขึ้นทำให้เราแก้ไขข้อผิดพลาดได้ยาก หรือแม้แต่เรื่องอาร์เรย์ก็ตาม ถ้าเราประกาศอาร์เรย์ไว้ 100 ไอเทม ถ้าบรรทัดใดในโค้ดของเราเกิดมีการอ้างถึงอาร์เรย์ขึ้นมา compiler ก็จะคอมไพล์ไม่ผ่านทันที ซึ่งจะกำหนดไว้ว่าตรงนี้เกิด Runtime error ได้ ตอนที่รัน นี่ก็ความฉลาดมากยิ่งขึ้นในตัว compiler ของ visual studio.net ซึ่งจะทำให้เราพัฒนาแอปพลิเคชันได้สะดวก โดยปราศจาก Runtime error มากที่สุด

ในเรื่องระบบรักษาความปลอดภัยนั้น ตัว .net ก็สามารถกำหนดได้ว่าผู้ใช้ที่จะรันแอปพลิเคชันของเรานั้นมีสถานะ (permission) เป็นอะไร อาจจะต้องการสถานะเป็น admin ถึงจะรันแอปพลิเคชันของเราได้

2.3.5 รูปแบบการทำงานร่วมกับภาษาอื่นๆ

การพัฒนาแอปพลิเคชันใน .net รวมไปถึงเทคนิคต่างๆที่กล่าวมาแล้วนั้น ไม่จำเป็นจะใช้เฉพาะภาษาตระกูลใน Visual Studio ไม่ว่าจะเป็น VB.net, C# หรือ C++ ถ้าภาษาใดก็ตามสามารถที่จะคอมไพล์มาให้เป็น Common Language Runtime ได้ ภาษานั้นก็จะสามารถใช้คุณลักษณะทั้งหมดของ .net ได้ ซึ่งตอนนี้มีภาษาอื่นๆที่สามารถทำงานร่วมกับแอปพลิเคชัน .net มากกว่า 20 ภาษาที่พัฒนาตัว compiler ขึ้นมาเพื่อคอมไพล์ภาษาต่างๆ ให้อยู่ในรูปแบบของแพลตฟอร์ม .net ไม่ว่าจะเป็นภาษา COBOL, PASCAL และอื่นๆ

ในตัว Visual Studio .net ก็มีเครื่องมือระดับสูง (High Level Tools) ที่จะทำให้เราพัฒนาแอปพลิเคชันได้สะดวกและง่ายมากขึ้น นี่คือเรื่องของ Common Language Runtime ซึ่งถือได้ว่าเป็นเลเยอร์ล่างสุดของการคอมไพล์แอปพลิเคชันที่เขียนด้วย Visual Studio.net

2.4 เลเยอร์ Base Class Library

เลเยอร์ถัดมาของโครงสร้างการพัฒนาแอปพลิเคชัน .net ต่อจาก Common Language Runtime ก็คือ Base Class Library เทคนิคต่างๆที่ Visual Studio.net จัดเตรียมให้เราในการใช้งานนั้น Base Class Library เปรียบเสมือน เป็นการรวบรวมเอาฟังก์ชันของ API(Application Programming Interface) ทั้งหมด สมมุติว่าตอนที่เราพัฒนาแอปพลิเคชันด้วย Visual Studio 6.0 เวลาเราจะเรียกใช้งานฟังก์ชันบางอย่างในระดับสูง หรือต้องการทำงานแบบลึกๆกับระบบ เรามักจะเรียกใช้ API นี้ ภาษาใน Visual Studio 6.0 มีความสามารถในการเรียกฟังก์ชัน API ได้ต่างกัน API บางตัวในปัจจุบันอาจจะต้องใช้ C++ ถึงจะเรียกได้ เพราะต้องใช้โครงสร้างข้อมูล (data structure) หรือ pointer อะไรบางอย่างที่ไม่มีใน Visual Basic แต่ API บางตัวก็สามารถเรียกจาก Visual Basic ได้

ตัว Base Class Library ก็คือ การที่เรารวบรวมฟังก์ชัน API ซึ่งกระจัดกระจายอยู่ เวลาจะเรียกใช้เราต้องไปค้นหาใน Help นั่นคือ Base Class Library พยายามที่จะรวบรวม API และ ฟังก์ชันทั้งหมดเกี่ยวกับระบบเข้ามาไว้ในลักษณะของ Object Oriented ทำเป็นคลาสอันหนึ่งซึ่งเป็นมาตรฐานเป็นคลาสที่สร้างมาในตัวระบบเรียบร้อยแล้ว ซึ่งคลาสทั้งหมดจะอยู่ภายใต้คลาสหลักอันหนึ่งที่เรียกว่า System

ทุกอย่างที่พัฒนาด้วยภาษาใน Visual Studio.net จะเป็น Object Oriented ทั้งหมด โดยมีคลาสที่ใหญ่ที่สุดโดยเรียกว่าคลาส System ซึ่งภายใต้คลาส System จะมีคลาสย่อยๆมากมาย ซึ่งแต่ละอันจะสนับสนุนฟังก์ชัน API หรือสนับสนุนการทำงานที่เราต้องการได้ ไม่ว่าจะเป็นเรื่องของการทำกราฟฟิก การทำเกี่ยวกับโครงสร้างข้อมูล (data structure) การทำเกี่ยวกับเรื่องเครือข่าย (network) ฟังก์ชัน API เหล่านี้จะถูกจัดกลุ่มให้เป็น Object Oriented อยู่ภายใต้ System Class การเรียกใช้งาน System Class ถ้าเป็น VB กับ C# ก็ใช้งานได้ทั้ง 2 อย่าง

เลเยอร์ที่ถูกส่งขึ้นมาจาก Base Class Library รวมทั้งเป็นแนวคิดซึ่งไม่โครซอฟท์ผลักดันมากคือเรื่องของ Web service นั่นเอง

2.5 เลเยอร์ Common Language Specification

เลเยอร์สุดท้ายในสถาปัตยกรรม .net ที่เราจะพูดถึงก็คือ Common Language Specification คือเรื่องของมาตรฐานภาษาบนพื้นฐาน .net ซึ่ง compiler จะต้อง ทำงานตาม มาตรฐานดังกล่าว เพื่อให้สามารถทำงานร่วมกับภาษาบนพื้นฐาน .net และภาษาอื่นๆได้ ไมโครซอฟท์ ได้ทำการปรับภาษาต่างๆเช่น C#, VB ให้เข้ามาตรฐาน .net นอกจากนั้น ผู้ผลิตรายอื่นสามารถพัฒนา ตามข้อกำหนดนี้เพื่อให้สามารถทำงานบนพื้นฐาน .net ได้

เลเยอร์ที่ 3 ของสถาปัตยกรรม .net ก็คือเครื่องมือที่ใช้ในการสร้างแอปพลิเคชัน หรือหลักการที่ ใช้ในการเขียนโปรแกรมต่างๆเช่นเรื่องของ ADO.net , ASP.net ที่ใช้ในการพัฒนาแอปพลิเคชัน แต่ สิ่งที่อยู่เหนือกว่าทุกอย่างก็คือภาษาที่เราใช้งาน ภาษาต่างๆที่ทำงานใน .net นั้นมีข้อดีคือ ต้อง สันนิษฐานมาตรฐานเดียวกัน เรียกว่า Common Language Specification ซึ่งไมโครซอฟท์ได้จัด ทะเบียนมาตรฐานนี้เข้ากับองค์กร ECMA แล้ว ซึ่งเป็นองค์กรที่ดูแลเรื่องโครงสร้างของภาษาต่างๆ และ ก็เป็นแบบเปิด(open) ด้วย เพราะฉะนั้นเจ้าของภาษาอื่นๆ ก็สามารถสร้างตัวแปลภาษา หรือ compiler เพื่อคอมไพล์ภาษาของเขาให้เข้ามาเป็น Common Language Specification อันนี้ได้ ในไม่ช้าเราจะ เห็นเว็บเพจเขียนด้วยภาษา COBOL ก็ได้ รวมทั้งภาษาอื่นๆด้วย นอกจากนี้ในตระกูล .net เองเราก็มี Visual Basic, Visual C++ กับ Visual C# ซึ่งภาษาอื่นๆ ที่เรากล่าวถึงนั้นปัจจุบันก็มีกว่า 20 ภาษา เช่น ภาษา MEL, COBOL, PASCAL, ไอเฟล , ไพทอน, Perl, Smalltalk ภาษา ML ซึ่งเป็นภาษา ที่ใช้งานกันในต่างประเทศ ภาษาประเภท Object ทั้งหมดสามารถทำเป็นแพลตฟอร์มของ .net ได้ เพราะว่าใน .net นั้นผลิตทุกอย่างเป็น Object Oriented ทั้งหมด